

Progetto del corso di Applicazioni Web

- A.A. 2025/26

1. Introduzione

Il progetto consiste nello sviluppo di **ConversAI**, un'applicazione web full-stack per la gestione di conversazioni testuali con un assistente simulato.

L'applicazione dovrà fornire un'interfaccia ispirata alle chat con assistenti basati su **Large Language Model**: una sidebar permetterà di visualizzare e selezionare i progetti disponibili e di accedere alle conversazioni associate, mentre l'area principale mostrerà la **conversazione corrente** e i relativi messaggi. L'utente dovrà poter creare nuovi progetti, aprire conversazioni esistenti, creare nuove conversazioni dedicate a specifici topic e inviare messaggi testuali tramite un form dedicato.

Ogni messaggio inviato dall'utente dovrà essere trasmesso al server tramite una richiesta **HTTP**. Il back-end dovrà salvare il messaggio nella conversazione corrente, generare una **risposta simulata** dell'assistente e restituire al front-end i dati necessari per aggiornare l'interfaccia. La conversazione dovrà quindi aggiornarsi dinamicamente, senza ricaricare la pagina.

Progetti, conversazioni e messaggi dovranno essere gestiti tramite **API REST** esposte da un server **Express.js** e persistiti in un database **SQLite**. L'applicazione dovrà quindi integrare in modo coerente interfaccia utente, logica lato server e archiviazione persistente dei dati.

2. Contesto e principi

Le applicazioni basate su **Large Language Model** utilizzano spesso interfacce conversazionali: l'utente invia un **prompt**, riceve una risposta e può continuare lo scambio con ulteriori messaggi. La conversazione diventa quindi una sequenza ordinata di interazioni, in cui ogni nuovo messaggio si aggiunge a quelli precedenti.

Alcuni strumenti recenti organizzano queste interazioni in **progetti** o spazi di lavoro. Ad esempio, la funzionalità **Projects** di ChatGPT raccoglie chat, file e istruzioni in uno stesso contesto di lavoro, mentre Claude offre uno strumento analogo per gestire workspace con cronologie di chat e basi di conoscenza dedicate [1, 2]. Questa organizzazione permette di distinguere attività diverse e di mantenere separati gli scambi legati a temi differenti.

ConversAI riprende questo modello in forma semplificata. Un **progetto** rappresenta un contenitore tematico. All'interno di un progetto, l'utente può creare più **conversazioni**, ciascuna dedicata a uno specifico **topic**. Ogni conversazione contiene una sequenza di **messaggi**, prodotti dall'utente e dall'assistente simulato.

La risposta dell'assistente viene generata lato server attraverso una funzione di **simulazione**. Dal punto di vista dell'applicazione, tale risposta dovrà essere gestita come un normale messaggio della conversazione, con un ruolo, un contenuto testuale e un riferimento alla conversazione a cui appartiene.

3. Obiettivo

L'obiettivo del progetto è realizzare una versione funzionante di **ConversAI**, composta da un'interfaccia web dinamica, un server **Express.js** e un database **SQLite**.

L'applicazione dovrà permettere all'utente di gestire tre risorse principali: **progetti**, **conversazioni** e **messaggi**. I progetti dovranno funzionare come contenitori tematici; le conversazioni dovranno appartenere a un progetto ed essere dedicate a uno specifico **topic**; i messaggi dovranno rappresentare gli scambi tra l'utente e l'assistente simulato.

Il front-end dovrà consentire all'utente di interagire con l'applicazione senza ricaricare la pagina: creare e selezionare progetti, aprire e gestire conversazioni, inviare prompt, visualizzare i messaggi e ricevere feedback sull'esito delle operazioni.

Il back-end dovrà esporre **API REST** per gestire le risorse dell'applicazione, validare i dati ricevuti, interagire con il database SQLite e restituire risposte in formato **JSON**. Le operazioni dovranno usare in modo coerente i metodi HTTP e restituire codici di stato appropriati.

La risposta dell'assistente dovrà essere generata lato server tramite una funzione di **simulazione** e salvata nel database come parte della conversazione. Il risultato finale dovrà essere una web application eseguibile localmente, con codice organizzato, dati persistenti e un'interfaccia chiara per la gestione delle conversazioni.

4. Cosa fare

Questa sezione descrive i requisiti operativi di **ConversAI**. Il progetto dovrà essere realizzato come applicazione web full-stack eseguibile in locale. Il front-end dovrà essere sviluppato con **HTML**, **CSS** e **JavaScript**, mentre il back-end dovrà essere sviluppato con **Node.js** ed **Express.js**. La persistenza dei dati dovrà essere gestita tramite **SQLite**.

Il progetto dovrà includere un file **package.json** con le dipendenze necessarie e uno script per avviare il server. L'applicazione dovrà funzionare senza richiedere servizi esterni, autenticazione o integrazione con un vero modello linguistico. Il codice dovrà essere organizzato in modo chiaro, separando per quanto possibile i file del **front-end**, il codice del **server** e il codice relativo all'accesso al **database**.

Le sottosezioni 4.1–4.4 specificano i requisiti **obbligatori per tutti**. Nei lavori di gruppo dovranno essere implementate almeno **due** tra le funzionalità aggiuntive descritte nelle sezioni 4.5–4.8. Le funzionalità scelte dovranno estendere il comportamento dell'applicazione e coinvolgere, quando necessario, front-end, back-end e persistenza dei dati.

4.1 Back-end e API REST

Il back-end dovrà essere realizzato con **Express.js** ed esporre un insieme di **API REST** per permettere al front-end di interagire con le risorse gestite dall'applicazione.

Le API dovranno restituire dati in formato **JSON** e usare in modo coerente i metodi HTTP. Gli endpoint dovranno essere progettati attorno alle risorse principali dell'applicazione: **progetti**, **conversazioni** e **messaggi**. La struttura dei path dovrà rendere chiaro se una richiesta riguarda una collezione di risorse, una singola risorsa, le conversazioni di un progetto o i messaggi di una conversazione.

Le API dovranno supportare le operazioni necessarie alla gestione dell'applicazione: creazione e recupero dei **progetti**; creazione e recupero delle **conversazioni**; recupero dei **messaggi** di una conversazione; invio di un nuovo messaggio da parte dell'utente. Le operazioni di **modifica** ed **eliminazione** dovranno essere implementate almeno per una tra le risorse principali dell'applicazione, ad esempio progetti o conversazioni.

La progettazione degli endpoint dovrà essere documentata nella relazione finale. Per ogni endpoint, la relazione dovrà indicare il **metodo HTTP**, il **path**, gli eventuali dati ricevuti, il tipo di risposta restituita e i principali casi di errore gestiti.

Le richieste dovranno essere validate lato server. Quando una richiesta non può essere completata, il server dovrà restituire un **codice di stato HTTP** appropriato e un messaggio di errore in formato **JSON**. Ad esempio, una richiesta con dati mancanti o non validi dovrà restituire un errore **400 Bad Request**, mentre una richiesta riferita a una risorsa inesistente dovrà restituire un errore **404 Not Found**.

4.2 Persistenza con SQLite

I dati di **ConversAI** dovranno essere persistiti in un database **SQLite**.

Il database dovrà contenere almeno tre tabelle: **projects**, **conversations** e **messages**, dedicate rispettivamente ai progetti, alle conversazioni e ai messaggi dell'applicazione.

Ogni progetto dovrà includere almeno un identificatore univoco, un nome, una descrizione opzionale, una data di creazione e una data di aggiornamento. Ogni conversazione dovrà includere almeno un identificatore univoco, il riferimento al progetto di appartenenza, un **topic**, una data di creazione e una data di aggiornamento. Ogni messaggio dovrà includere almeno un identificatore univoco, il riferimento alla conversazione di appartenenza, un **ruolo**, un contenuto testuale e una data di creazione. Il ruolo dovrà distinguere i messaggi inviati dall'utente da quelli generati dall'assistente simulato.

Ogni conversazione dovrà appartenere a un progetto, e ogni messaggio dovrà appartenere a una conversazione. Questa organizzazione dovrà essere rispettata sia nel database SQLite sia nelle risposte **JSON** restituite dal server.

Il server dovrà leggere e scrivere i dati attraverso il database **SQLite**. Le operazioni svolte tramite le **API** dovranno riflettersi sui **dati persistenti**: la creazione di una risorsa dovrà

aggiungere nuovi dati, la lettura dovrà recuperare i dati richiesti, mentre le operazioni di modifica ed eliminazione, quando previste, dovranno aggiornare lo stato salvato nel database. Non è richiesto l'uso di funzionalità **SQL** avanzate; è sufficiente usare operazioni SQL di base per inserire, recuperare, aggiornare o rimuovere i dati gestiti dal server.

Il database dovrà essere inizializzato dal progetto stesso. È possibile predisporre uno script dedicato, oppure creare le tabelle all'avvio dell'applicazione con **CREATE TABLE IF NOT EXISTS**. In entrambi i casi, la relazione dovrà contenere le istruzioni necessarie per ricreare correttamente il database in locale.

4.3 Interfaccia utente

Il front-end dovrà fornire un'interfaccia web dinamica per utilizzare **ConversAI** dal browser. L'utente dovrà poter navigare tra i progetti, aprire le conversazioni associate e inviare nuovi messaggi senza ricaricare la pagina.

L'interfaccia dovrà includere almeno una **sidebar** e un'area principale. La sidebar dovrà permettere di visualizzare e selezionare i progetti disponibili e di accedere alle conversazioni associate al progetto selezionato. L'area principale dovrà mostrare la conversazione corrente, il relativo **topic**, lo storico dei messaggi e un form per l'invio di un nuovo **prompt**.

I **messaggi** dovranno essere visualizzati in modo ordinato e distinguibile. L'interfaccia dovrà rendere chiaro quali messaggi sono stati inviati dall'utente e quali sono stati generati dall'assistente simulato.

Il front-end dovrà comunicare con il back-end tramite **Fetch API**. Le risposte ricevute dal server dovranno essere usate per aggiornare il **DOM** e mantenere l'interfaccia coerente con i dati salvati nel database.

L'interfaccia dovrà fornire un **feedback** comprensibile all'utente durante le operazioni principali, ad esempio in caso di creazione, modifica o eliminazione quando previste, invio di un messaggio, dati mancanti o errori restituiti dal server. L'applicazione dovrà essere leggibile, usabile e organizzata in modo coerente.

4.4 Simulazione delle risposte

Il progetto non richiede l'integrazione con un servizio esterno o con un vero modello linguistico. Le risposte dell'assistente dovranno essere generate dal server attraverso una funzione di **simulazione**.

Quando l'utente invia un nuovo **prompt**, il server dovrà salvarlo come messaggio dell'**utente** nella conversazione corrente. Successivamente, dovrà generare una risposta simulata, salvarla come messaggio dell'**assistente** e restituire al front-end i dati necessari per aggiornare l'interfaccia.

La risposta simulata dovrà essere prodotta **lato server** e dovrà permettere di verificare il funzionamento completo dell'applicazione: invio del prompt, elaborazione della richiesta,

salvataggio dei messaggi, recupero della conversazione aggiornata e visualizzazione nel front-end.

La funzione di simulazione potrà basarsi su regole semplici, ad esempio usando il contenuto del prompt, il **topic** della conversazione o alcuni template predefiniti. La risposta non dovrà simulare le capacità di un vero modello linguistico, ma dovrà mostrare che il server elabora il prompt e produce un messaggio dell'assistente non completamente fisso.

4.5 Ricerca nello storico

Se scelta, questa funzionalità dovrà permettere all'utente di effettuare ricerche nello **storico delle conversazioni**. La ricerca potrà riguardare, ad esempio, il contenuto dei messaggi, il topic delle conversazioni o i nomi dei progetti.

La funzionalità dovrà prevedere un'interazione lato front-end, una richiesta al back-end e il recupero dei risultati dal database **SQLite**. I risultati dovranno essere mostrati nell'interfaccia in modo chiaro, permettendo all'utente di capire a quale progetto, conversazione o messaggio si riferiscono.

La relazione dovrà descrivere quali dati vengono considerati nella ricerca, come viene inviata la richiesta al server e come vengono visualizzati i risultati nel front-end.

4.6 Conversazioni preferite o archiviate

Se scelta, questa funzionalità dovrà permettere all'utente di organizzare le conversazioni marcandole come **preferite** o **archivate**.

Le conversazioni preferite potranno essere mostrate in evidenza nell'interfaccia, mentre le conversazioni archiviate potranno essere separate o nascoste dalla vista principale. La scelta precisa dell'interazione è lasciata al gruppo, purché il comportamento sia chiaro e coerente.

La funzionalità dovrà essere gestita dal back-end e salvata nel database **SQLite**. L'interfaccia dovrà aggiornarsi in modo dinamico quando una conversazione viene marcata, rimossa dai preferiti, archiviata o ripristinata.

4.7 Esportazione di una conversazione

Se scelta, questa funzionalità dovrà permettere all'utente di esportare una **conversazione completa**, includendo il topic e lo storico dei messaggi.

L'esportazione potrà produrre un contenuto in formato testuale, ad esempio testo semplice o **JSON**. Il formato scelto dovrà essere adatto a rappresentare in modo leggibile la sequenza dei messaggi e il ruolo di ciascun messaggio.

La funzionalità dovrà recuperare i dati necessari dal back-end e renderli disponibili all'utente attraverso l'interfaccia. La relazione dovrà spiegare il formato scelto per l'esportazione e il flusso seguito per generare il contenuto esportato.

4.8 Template di prompt

Se scelta, questa funzionalità dovrà permettere all'utente di creare e utilizzare **template di prompt** riutilizzabili.

Un template dovrà rappresentare una traccia di prompt che l'utente può selezionare e inserire nel form di invio. Ad esempio, un template potrebbe servire per chiedere una spiegazione, una sintesi, un esempio o una riformulazione.

La funzionalità dovrà prevedere un'interazione lato front-end, la gestione dei template tramite back-end e la persistenza dei dati nel database **SQLite**. L'interfaccia dovrà permettere all'utente di visualizzare i template disponibili e usarli per preparare un nuovo **prompt**.

La relazione dovrà descrivere come vengono rappresentati i template, quali operazioni sono disponibili e come possono essere utilizzati durante una conversazione.

5. Come cominciare

Gli studenti devono organizzarsi in **gruppi di tre persone**. È ammesso anche il lavoro individuale, ma non sono accettati gruppi con un numero diverso di componenti.

Per iniziare, ogni gruppo o studente singolo deve comunicare via email la propria intenzione di svolgere il progetto, prima di avviare lo sviluppo.

La mail va inviata a **michael.soprano@uniud.it** con il seguente oggetto:

[Progetto WebApp - 2025/2026 - Richiesta] cognome1[_cognome2_cognome3]

Nel corpo del messaggio è sufficiente indicare i componenti del gruppo, riportando nome, cognome e numero di matricola di ciascuno. Tutti i membri del gruppo devono essere in copia alla mail.

La conferma sarà comunicata via email entro pochi giorni. La data di risposta da parte del docente costituisce il riferimento per il calcolo dei **45 giorni** disponibili per la consegna. Il progetto può essere avviato solo dopo la conferma ufficiale.

6. Come consegnare

La consegna deve avvenire entro **45 giorni** dalla data di conferma del progetto da parte del docente, comunicata via email in risposta alla richiesta inviata dal gruppo o dallo studente singolo.

Ciascun gruppo, o studente singolo, dovrà consegnare una relazione in formato **PDF** di massimo **6 pagine**, redatta in modo chiaro e sintetico, contenente nomi, cognomi e numeri di matricola di tutti i partecipanti. La relazione dovrà descrivere le principali scelte progettuali adottate nello sviluppo di **ConversAI**, includendo almeno l'architettura dell'applicazione, la struttura del database, gli endpoint **REST** progettati, il flusso di comunicazione tra front-end e back-end e la funzione usata per simulare le risposte dell'assistente. Eventuali funzionalità aggiuntive, limitazioni note o scelte alternative rispetto ai requisiti dovranno essere motivate chiaramente.

La consegna dovrà includere anche il codice sorgente completo dell'applicazione, un file **README.md** ed eventuali materiali aggiuntivi utili alla comprensione del progetto, come screenshot, diagrammi o esempi di richieste e risposte. Il codice dovrà comprendere il front-end, il back-end, l'accesso al database **SQLite**, il file **package.json** e tutti i file necessari per eseguire l'applicazione. Il file **README.md** dovrà contenere le istruzioni operative per installare le dipendenze, inizializzare il database e avviare il progetto in locale.

Tutto il materiale va caricato in una repository pubblica su **GitHub**. Questa modalità di consegna evita problemi legati ai limiti degli allegati email e permette di rendere disponibili in modo ordinato tutti i file necessari per valutare il progetto. Il gruppo dovrà verificare che la repository sia visibile pubblicamente, ad esempio aprendo il link in una finestra in incognito.

Per finalizzare la consegna, ogni gruppo, o studente singolo, deve inviare una mail a **michael.soprano@uniud.it** con il seguente oggetto:

[Progetto WebApp - 2025/2026 - Consegna] cognome1[_cognome2_cognome3]

Nel corpo del messaggio è sufficiente inserire il link alla repository GitHub contenente tutto il materiale. La mail deve essere messa in copia a tutti i membri del gruppo.

È responsabilità del gruppo verificare con attenzione che il link fornito sia corretto e funzionante, che la repository sia accessibile, che la consegna sia completa e che tutto sia inviato entro i termini stabiliti.

Consegne incomplete, inaccessibili o oltre la scadenza non saranno prese in considerazione.

7. Discussione

Dopo la consegna, ogni gruppo o studente singolo sarà convocato per una breve discussione sul progetto. La data sarà comunicata via email dopo la ricezione del materiale.

Durante la discussione sarà richiesto di illustrare sinteticamente le funzionalità principali sviluppate e di rispondere a domande relative alle **scelte progettuali**, alla **struttura del codice** e al **funzionamento dell'applicazione**.

La discussione potrà includere anche domande generali sugli argomenti del corso collegati al progetto, come **HTTP**, **API REST**, interazione front-end/back-end, gestione del **DOM**, uso di **Fetch API**, **Express.js** e persistenza dei dati.

La discussione rappresenta una parte integrante della **valutazione finale**. Tutti i membri del gruppo dovranno partecipare ed essere in grado di spiegare, anche individualmente, il proprio contributo e le decisioni prese durante lo sviluppo.

Bibliografia

- [1] OpenAI. *Projects in ChatGPT*. OpenAI Help Center. Consultato il 18 maggio 2026.
<https://help.openai.com/en/articles/10169521-projects-in-chatgpt>
- [2] Anthropic. *What are projects?* Claude Help Center. Consultato il 18 maggio 2026.
<https://support.claude.com/en/articles/9517075-what-are-projects>